

Testing

Cohort 2- Group 1

Members

Ollie Barlow (vrq506)
Jacob Barnes (bdp514)
Zayaan Bawany (fst508)
Lily Boldbaatar (jkk522)
Georgia Briggs (rxm503)
Evelyn Gravett (qnf502)
Hassan Yunus (wfv505)

Test Summary

We implemented our tests with JUnit and also used Mockito to mimic objects, for example the BuildingTests.java file uses mocked textures to replicate UI elements so we could isolate the underlying logic. We used a narrow test scope (unit tests) to test small parts of the code base e.g. singular classes so it is easier and faster to run and there are fewer things to check if the tests fail. Unit tests isolate specific functionalities, classes and methods making it easier to detect faults and failures in the software. We did not include any integration or end to end test. We used automated tests as they lead to improved documentation, thoughtful design and less debugging. We also used edge case testing to ensure our game handles both expected and unexpected behavior such as the user trying to place a building outside the boundaries of the game, also preventing the building being deleted or misaligned with the grid implemented in the game

Test Results

ASSET

We ran a test to ensure that all assets were loaded correctly and are present when the game starts.

testAssetsExist() confirms that all required assets load without failures or errors. The test was passed meaning assets are working correctly and are reliable.

Related Requirements: UR_GRAPHICS

BUILDING

A test suite to ensure the correct behaviour of buildings in the game and make sure there is proper interaction with the map. The key tests are:

testHandleDragging() ensures the snapping of building to the map grid when it is being dragged to be placed

testClampPosition() ensures buildings stay within the boundaries of the map, clamping the building to the map edge when the user mouse position is out of the boundaries of the game

testSelection() tests the ability to select different buildings from the building menu

testPlacement() confirms the correct placement of buildings and also the deselection of buildings

testGetDistanceFromBuilding() calculates the distance between two buildings

Related Requirements: UR_MAP, UR_BUILD, UR_GAMEPLAY

SATISFACTION

A test suite to test the satisfaction system, which changes based on thoughts and behaviors.

testSetGetThought() ensures thoughts can be added and retrieved

testRemoveThought() makes sure thoughts can be removed

testMinTarget() tests the lower bound of satisfaction making sure it doesn't go below 100

testMaxTarget() tests upper bound of satisfaction making sure it doesn't go above 100

testManyThoughtTarget() ensures that the calculation of multiple thoughts is correct

testUpdateScore() tests if the satisfaction scores increment/decrement towards the target

Related Requirements: UR_SCORE_SYSTEM, UR_GAMEPLAY

Event

A test suite to ensure events are handled correctly, both simple and conditional events.

testSimpleThoughtEvent() ensures events trigger at the times they were specified

testConditionThoughtEvent() tests if conditional events work correctly based on certain states in the game

Related Requirements: UR_EVENTS

LEADERBOARD

A test suite to ensure leaderboard functionality for tracking and ranking scores

testAddScore() ensures scores can be added and that they are not overwritten

testClear() verifies the leaderboard can be cleared of all previous scores

testSortScore() ensures scores are sorted in descending order

Related Requirements: UR_LEADERBOARD, UR_SCORE_SYSTEM

ACHIEVEMENT

A test suite that ensures the tracking and execution of player achievements

testAchievementCompletion() makes sure that achievements are moved from incomplete to complete when the user meets the conditions

testAchievementOnComplete() ensures actions tied to the achievements are successfully executed on completion

Related Requirements: UR_ACHIEVEMENTS

We tested all major parts of our implementation and all tests came back successful suggesting the implementation follows specific behaviors. The tests we have conducted show a high level of completeness as they cover the highest priority and core functions of the game. There are also edge case tests that test the boundaries of the game which help completeness, however we did not cover some areas, e.g. UR_BACKGROUND_MUSIC. Since we used JUnit assertions across the whole test suite, we ensured the validation of

expected outcome. All tests also passed with no errors, showing the correctness of the game logic.

Test Summary

20

tests

0

failures

0

ignored

1.854s

duration

100%

successful

Packages

Classes

Class	Tests	Failures	Ignored	Duration	Success rate
com.badlogic.UniSim2.headless.AchievementTests	2	0	0	1.131s	100%
com.badlogic.UniSim2.headless.AssetTests	1	0	0	0.360s	100%
com.badlogic.UniSim2.headless.BuildingTests	6	0	0	0.337s	100%
com.badlogic.UniSim2.headless.EventTests	2	0	0	0.005s	100%
com.badlogic.UniSim2.headless.LeadersboardTests	3	0	0	0.016s	100%
com.badlogic.UniSim2.headless.SatisfactionTests	6	0	0	0.005s	100%